# Enunciado

O Conway's Game of Life é um exemplo de um autómato celular. Neste problema modifica-se as regras do autómato de forma a usar um espaço de estados finito.

1. O espaço de estados é definido por uma grelha de células booleanas (morta = 0/viva = 1) de dimensão $N \times N$ (com $N > 16$) identificadas por índices $(i, j) \in \{1..N\}$. Estas $N^2$ células são aqui referidas como "normais".
2. Inicialmente todas as células normais estão mortas excepto as células $i, j \leq 3$ que estão vivas. Um estado onde todas as células normais estão mortas é um "estado de erro".
3. Adicionalmente existem :
4. $2N + 1$ "células iniciais" que correspondem a um dos índices, $i$ ou $j$, ser zero. As células iniciais têm valores constantes que, no estado inicial, são gerados aleatoriamente com uma probabilidade $1/2$ de estarem vivas.
5. $2N + 3$ "células finais" que não são iniciais e um dos seus índices tem o valor $N + 1$. As células finais têm valores constantemente $0$; i.e. estão mortas.
6. As células normais do autómato modificam o estado de acordo com a regra "B3|S23": i.e. a célula nasce (passa de $0$ a $1$) se tem exatamente 3 vizinhos vivos; sobrevive (mantém-se viva) se o número de vizinhos vivos é 2 ou 3 ; caso contrário morre ou continua morta.
7. A célula $(i_0, j_0)$ e $(i_1, j_1)$ são vizinhas sse $\max(|i_0 - i_1|, |j_0 - j_1|) = 1$

Pretende-se:

1. Construir uma máquina de estados finita (FSM) que represente este autómato. Este problema tem duas interpretações possíveis para as transições da FSM

2. Em cada transição do sistema uma e só uma célula normal transita de estado.

3. Em cada transição do sistema todas as células normais transitam de estado simultaneamente. Ambas as opções devem ser contempladas e a relação de transição selecionada é determinada pelo valor de um parâmetro definido no código.

4. Provar (ou obter contra-exemplos) as seguintes propriedades:

5. Nunca se alcança o estado de erro

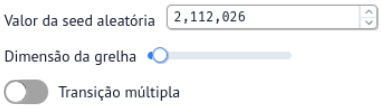
6. Nenhuma célula normal está permanentemente morta ou permanentemente viva.

# Resolução

In [ ]:
```
import marimo as mo
import random
import z3
```

In [ ]:
```
N_slide = mo.ui.slider(
    start=17, stop=100, value=20, label="Dimensão da grelha"
)
transicao_multipla = mo.ui.switch(label="Transição múltipla")
seed = mo.ui.number(value=2112026, label="Valor da seed aleatória")
```

In [ ]:
```
mo.vstack([seed, N_slide, transicao_multipla])
```




In [ ]:
```
N = N_slide.value
```

# Representação da máquina de estados em Z3

## Declaração do estado

Começamos por criar um a grelha inicial $(N+2)^2$, com as fronteiras constantes. As células inicias são geradas aleatoriamente com base em `seed`. As células finais são obrigatoriamente mortas. A verificação das células finais deve ocorrer primeiro para garantir que as células nos cantos extremos superior direito e inferior esquero estejam mortas.

In [ ]:
```
def criar_estado(t):
    rng = random.Random(seed.value)
    e = {}
    for r in range(N + 2):
        for c in range(N + 2):
            if r == N + 1 or c == N + 1:
                # 2N + 3 celulas finais sempre mortas
                e[(r, c)] = z3.BoolVal(False)

            elif r == 0 or c == 0:
                # 2N + 1 celulas iniciais com probabilidade 1/2 de estarem
vivas ou mortas
                viva = rng.choice([True, False])
                e[(r, c)] = z3.BoolVal(viva)

            else:
                # N * N células normais variadas no período t
                e[(r, c)] = z3.Bool(f"c_{t}_{r}_{c}")
```

```
    return e
```

## Estado inicial e estado de erro

Conforme especificado no enunciado, o estado inicial $e_0$ contém todas as células normais mortas, com exceção daquelas com ambos os índices $\leq 3$. O estado de erro é definido como o estado em que todas as célulsa normais estão mortas.

Definimos também uma função `estados_iguais` que verifica se dois estados possuem exatamente os mesmos valores para cada célula.

In [ ]:
```
def estado_inicial(e):
    return z3.And(
        [
            e[(r, c)] == (r <= 3 and c <= 3)
            for r in range(1, N + 1)
            for c in range(1, N + 1)
        ]
    )

def estado_erro(e):
    return z3.And(
        [
            z3.Not(e[(r, c)])
            for r in range(1, N + 1)
            for c in range(1, N + 1)
        ]
    )

def estados_iguais(e0, e1):
    return z3.And(
        [
            e0[(r, c)] == e1[(r, c)]
            for r in range(1, N + 1)
            for c in range(1, N + 1)
        ]
    )
```

## Transições

A regra de transição (B3|S23) especifica o comportamento de cada célula individual, definindo as condições necessárias para que uma célula nasça e continue viva. Definimos então o comportamento de cada passo de transição "ponto-a-ponto":

In [ ]:
```
def b3_s23(e, r, c):
    vizinhanca = [
        e[(r + dr, c + dc)]
        for dr in [-1, 0, 1]
        for dc in [-1, 0, 1]
        if dr != 0 or dc != 0  # nao coletar a si mesma
```

```
    ]

    # assistência de LLM (https://share.gemini.google/pGKKl1QqwsYj): uso de
PbEq
    # nasce ou sobrevive se tiver 3 vizinhos; sobrevive se tiver dois
vizinhos
    return z3.Or(
        z3.PbEq([(v, 1) for v in vizinhanca], 3),
        z3.And(e[(r, c)], z3.PbEq([(v, 1) for v in vizinhanca], 2)),
    )

    # código original
    ## nao há conversao automatica Bool -> Int
    # vizinhos_vivos = z3.Sum(z3.If(v, 1, 0) for v in vizinhanca)

    # return z3.Or(
    #     vizinhos_vivos == 3, z3.And(e[(r, c)], vizinhos_vivos == 2)
    # )
```

Podemos então definir a relação de transição entre dois estados $e_0$ e $e_1$:

- No caso de múltiplas transições, a condição de cada célula do estado $e_1$ deve representar a situação daquela mesma célula no estado $e_0$ depois da aplicação da regra B3|S23.
- No caso de transições singulares, precisamos verificar que $e_1$ só difere de $e_0$ a menos da situação de uma única célula.

In [ ]:
```
def transicao(e0, e1, multipla=True):
    if multipla:
        return z3.And(
            [
                e1[(r, c)] == b3_s23(e0, r, c)
                for r in range(1, N + 1)
                for c in range(1, N + 1)
            ]
        )
    else:
        # todas as celulas do estado mais recente mantem seu valor, ou
atualizam o valor do estado passado
        celulas_validas = [
            z3.If(
                e1[(r, c)] != e0[(r, c)],
                e1[(r, c)] == b3_s23(e0, r, c),
                True,
            )
            for r in range(1, N + 1)
            for c in range(1, N + 1)
        ]

        num_mudancas = z3.Sum(
            [
                z3.If(e1[(r, c)] != e0[(r, c)], 1, 0)
                for r in range(1, N + 1)
                for c in range(1, N + 1)
```

```
            ]
        )

    return z3.And(num_mudancas == 1, *celulas_validas)
```

In [ ]:
```
def desenha_estado(e, m=None):
    linhas = []
    for r in range(N + 2):
        linha = []
        for c in range(N + 2):
            val = (
                e[(r, c)]
                if m is None
                else m.eval(e[(r, c)], model_completion=True)
            )
            viva = z3.is_true(z3.simplify(val))
            linha.append("V" if viva else "M")
        linhas.append(" ".join(linha))
    return "\n".join(linhas)
```

# Propriedades

Seguimos agora com a verificação formal a partir de provas ou contra-exemplos das propriedades explicitadas no enunciado.

## 1: Nunca se alcança o estado de erro

Podemos interpretar essa propriedade em lógica temporal por $G\,\neg(\text{estado_erro})$.

- no passo $k$, verificamos se `estado_erro(estados[k])` é satisfazível. Se for, existe um caminho válido $e_0 \to \cdots \to e_k$ onde todas as células normais estão mortas em $e_k$, i.e., a propriedade é falsa e temos um contra-exemplo.

- se `estado_erro(estados[k])` for insatisfazível, então testamos se os estados no traço até $k$ são todos dois-a-dois distintos, i.e., $\bigwedge_{0 \le i < j \le k} \neg\text{estados_iguais}(e_i, e_j)$. Se esta restrição for insatisfazível, então sabemos que para o caso de transições múltiplas, numa execução de comprimento $k$ um estado anterior é obrigatoriamente repetido (formando um loop). Mas como já verificamos que nenhum dos estados anteriores até $k-1$ atinge o estado de erro, fica demonstrado que o estado de erro é inalcançável para qualquer $t \geq 0$, provando a propriedade.

Se nenhum dos dois casos se verificar, então aumentamos a profundidade de busca.

In [ ]:
```
def p1(multipla, profundidade):
    P = lambda e: z3.Not(estado_erro(e))

    s = z3.Solver()
    estados = [criar_estado(0)]
```

```
    s.add(estado_inicial(estados[0]))

    for k in range(profundidade + 1):
        if k > 0:
            estados.append(criar_estado(k))
            s.add(transicao(estados[-2], estados[-1], multipla=multipla))

        # base (bmc até k): verifica se ~P(e_k) é sat
        if s.check(z3.Not(P(estados[k]))) == z3.sat:
            return "refutada", estados, s.model()

        # testar se todos estados são distintos
        if k > 0:
            distintos = [
                z3.Not(estados_iguais(estados[i], estados[j]))
                for i in range(k + 1)
                for j in range(i + 1, k + 1)
            ]
            if s.check(*distintos) == z3.unsat:
                s.check()
                return "provada", estados, s.model()

    return "inconclusiva", estados, None

profundidade_maxima_p1 = mo.ui.slider(
    start=1,
    stop=400,
    value=200,
    label="Profundidade máxima para a propriedade 1",
)
profundidade_maxima_p1
```

Profundidade máxima para a propriedade 1

In [ ]:
```
res_p1, estados_p1, mod_p1 = p1(
    transicao_multipla.value, profundidade_maxima_p1.value
)

passo_p1 = mo.ui.slider(
    start=0,
    stop=len(estados_p1) - 1,
    value=len(estados_p1) - 1,
    step=1,
    label="t",
)
```

In [ ]:
```
mo.vstack(
    [
        mo.md(f"Resultado: {res_p1} (em $t = {len(estados_p1) - 1}$)"),
        *(
            [
                passo_p1,
                mo.md(
                    f"```\nt =
{passo_p1.value}:\n{desenha_estado(estados_p1[passo_p1.value],
```

```
mod_p1)}\n```"
                    ),
                ]
                if mod_p1 is not None
                else []
            ),
        ]
)
```

Resultado: refutada (em $t = 9$)

t

```
t = 9:
V M V M M M V M M M V M V V V V V M V M M
V M M M M M M M M M M M M M M M M M M M
M M M M M M M M M M M M M M M M M M M M
M M M M M M M M M M M M M M M M M M M M
V M M M M M M M M M M M M M M M M M M M
V M M M M M M M M M M M M M M M M M M M
V M M M M M M M M M M M M M M M M M M M
V M M M M M M M M M M M M M M M M M M M
M M M M M M M M M M M M M M M M M M M M
V M M M M M M M M M M M M M M M M M M M
V M M M M M M M M M M M M M M M M M M M
M M M M M M M M M M M M M M M M M M M M
M M M M M M M M M M M M M M M M M M M M
M M M M M M M M M M M M M M M M M M M M
M M M M M M M M M M M M M M M M M M M M
M M M M M M M M M M M M M M M M M M M M
V M M M M M M M M M M M M M M M M M M M
V M M M M M M M M M M M M M M M M M M M
M M M M M M M M M M M M M M M M M M M M
M M M M M M M M M M M M M M M M M M M M
M M M M M M M M M M M M M M M M M M M M
M M M M M M M M M M M M M M M M M M M M
```

## 2: Nenhuma célula normal está permanentemente morta ou permanentemente viva

Podemos interpretar em lógica temporal por $\neg(G\,\neg c \vee G\,c)$ para cada célula normal $c$ .

- no passo $k$, procuramos um um caminho $e_0 \to \cdots \to e_k$ que fecha um ciclo $\bigvee_{0 \le m < k} \text{estados_iguais}(e_m, e_k)$ onde alguma célula normal manteve o seu valor inicial em todos os passos, i.e., $\bigvee_c \bigwedge^k (e_0[c] = e_t[c])$. Se for satisfazível, temos um exemplo de uma execução que se repete na qual a célula permanece constante, i.e., a propriedade é falsa e temos um contra-exemplo.
- se for insatisfazível, testamos se os estados no traço até $k$ são todos dois-a-dois distintos, i.e., $\bigwedge_{0 \le i < j \le k} \neg\text{estados_iguais}(e_i, e_j)$. se a restrição for insatisfazível, todas as execuções de comprimento $k$ já repetiram obrigatoriamente um estado anterior sem que nenhum ciclo mantivesse uma célula permanente, ficando a propriedade provada.

Se nenhum dos dois casos se verificar novamente aumentmaos a profundidade da busca

In [ ]:
```
def p2(multipla, profundidade):
    s = z3.Solver()
    estados = [criar_estado(0)]
    s.add(estado_inicial(estados[0]))
```

```
    for k in range(1, profundidade + 1):
        estados.append(criar_estado(k))
        s.add(transicao(estados[-2], estados[-1], multipla=multipla))

        # tentamos detectar um ciclo para o contra-exemplo
        fecho_ciclo = z3.Or(
            [estados_iguais(estados[k], estados[m]) for m in range(k)]
        )

        # verificamos se existe alguma célula que permaneceu constante ao
longo de todos os estados do ciclo
        celula_permanente = z3.Or(
            [
                z3.And(
                    [
                        estados[t][(r, c)] == estados[0][(r, c)]
                        for t in range(1, k + 1)
                    ]
                )
                for r in range(1, N + 1)
                for c in range(1, N + 1)
            ]
        )

        if s.check(fecho_ciclo, celula_permanente) == z3.sat:
            return "refutada", estados, s.model()

        distintos = [
            z3.Not(estados_iguais(estados[i], estados[j]))
            for i in range(k + 1)
            for j in range(i + 1, k + 1)
        ]
        if s.check(*distintos) == z3.unsat:
            s.check()
            return "provada", estados, s.model()

    return "inconclusiva", estados, None

profundidade_maxima_p2 = mo.ui.slider(
    start=1,
    stop=400,
    value=200,
    label="Profundidade máxima para a propriedade 2",
)
profundidade_maxima_p2
```




In [ ]:

```
res_p2, estados_p2, mod_p2 = p2(
    transicao_multipla.value, profundidade_maxima_p2.value
)

passo_p2 = mo.ui.slider(
    start=0,
```

```
    stop=len(estados_p2) - 1,
    value=len(estados_p2) - 1,
    step=1,
    label="t",
)
```

In [ ]:

```
mo.vstack(
    [
        mo.md(f"Resultado: {res_p2} (em $t = {len(estados_p2) - 1}$)"),
        *(
            [
                passo_p2,
                mo.md(
                    f"```\nt =
{passo_p2.value}:\n{desenha_estado(estados_p2[passo_p2.value],
mod_p2)}\n```"
                ),
            ]
            if mod_p2 is not None
            else []
        ),
    ]
)
```

Resultado: refutada (em $t = 6$)

t

```
t = 6:
V M V M M M M V M M M V M V V V V V M V M M
V V V V M M M M M M M M M M M M M M M M M M
M V V V M M M M M M M M M M M M M M M M M M
M V V V M M M M M M M M M M M M M M M M M M
V M M M M M M M M M M M M M M M M M M M M M
V M M M M M M M M M M M M M M M M M M M M M
V M M M M M M M M M M M M M M M M M M M M M
V M M M M M M M M M M M M M M M M M M M M M
M M M M M M M M M M M M M M M M M M M M M M
V M M M M M M M M M M M M M M M M M M M M M
V M M M M M M M M M M M M M M M M M M M M M
M M M M M M M M M M M M M M M M M M M M M M
M M M M M M M M M M M M M M M M M M M M M M
M M M M M M M M M M M M M M M M M M M M M M
M M M M M M M M M M M M M M M M M M M M M M
M M M M M M M M M M M M M M M M M M M M M M
V M M M M M M M M M M M M M M M M M M M M M
V M M M M M M M M M M M M M M M M M M M M M
M M M M M M M M M M M M M M M M M M M M M M
M M M M M M M M M M M M M M M M M M M M M M
M M M M M M M M M M M M M M M M M M M M M M
M M M M M M M M M M M M M M M M M M M M M M
```